

Training Deep Learning Models

Dr. Parham Kebria

parhamkebria@ieee.org

Advanced Study Institute: Artificial Intelligence for Disaster Management
Orlando, November 17 – 25, 2025



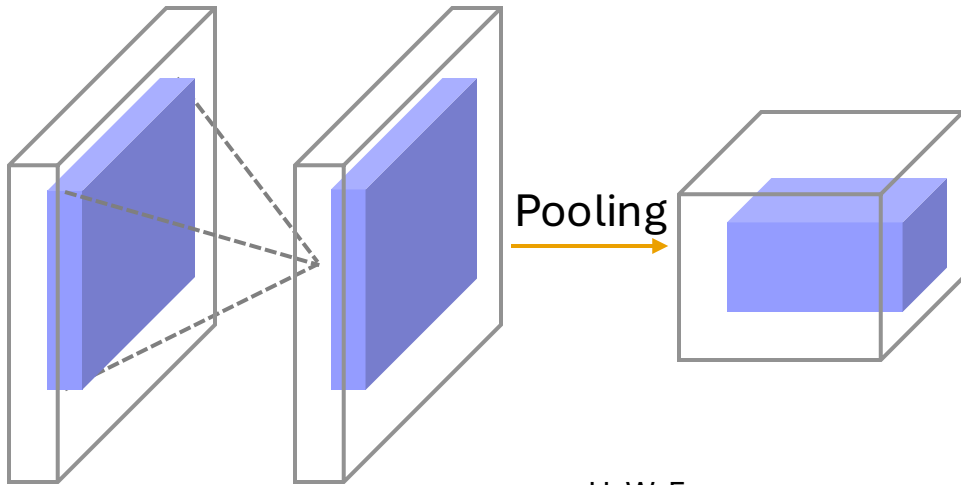
*This activity
is supported by:*

The NATO Science for Peace
and Security Programme

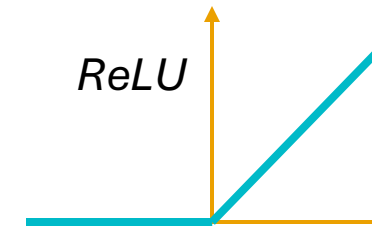
How to build and train *Deep* CNNs?

- Normalizations & Dropout
- Activation Functions
- Model Architectures
- Data Preprocessing
- Data Augmentation
- Transfer Learning
- Hyperparameter Tuning
- Code Practice

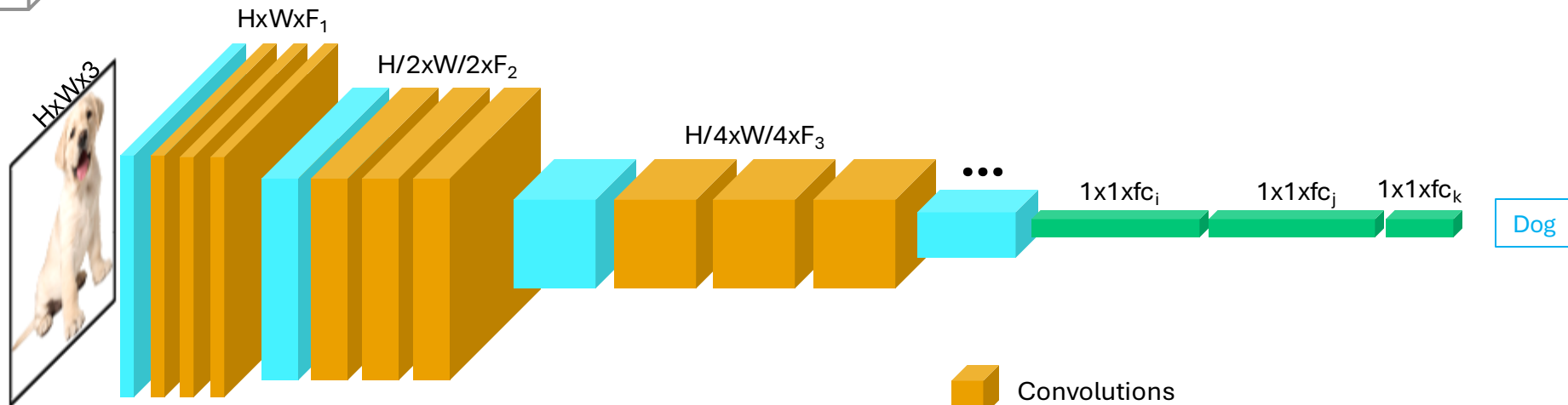
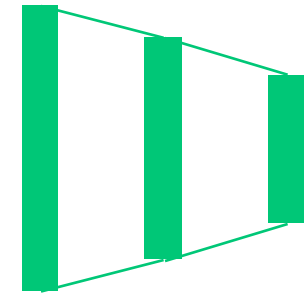
Convolution layers



Activation function



Fully-Connected (FC)



-  Convolutions
-  Subsampling (Max/Avg Pooling)
-  Fully connected

Training Deep Neural Networks

- Normalize input tensors/features
- *Scale/shift* by trainable parameters

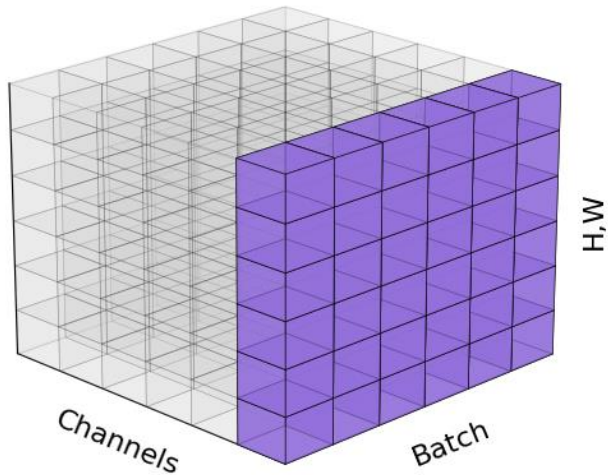
Normalization Layer

Normalize per batch $\rightarrow$

Trainable parameters $\rightarrow$

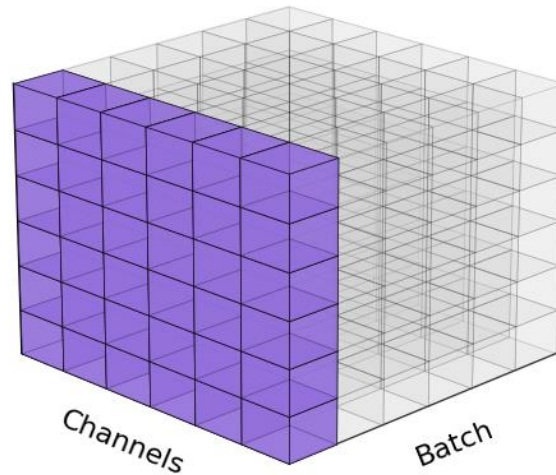
$$x: N \times D$$
$$\mu, \sigma: N \times 1$$
$$\beta: 1 \times D$$
$$y = \frac{\gamma(x - \mu)}{\sigma} + \beta\gamma$$

Batch Normalization

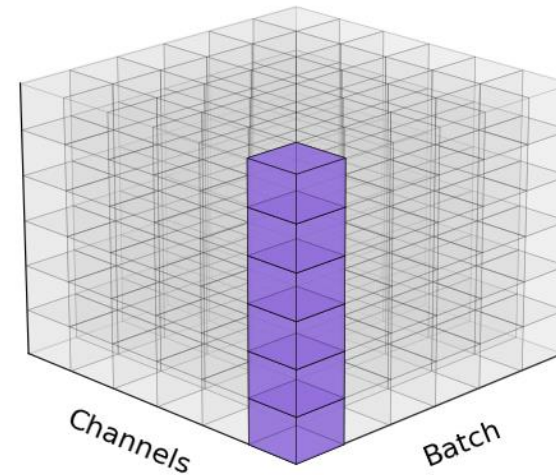


Layer Normalization

Ba, Kiros, & Hinton, arXiv 2016

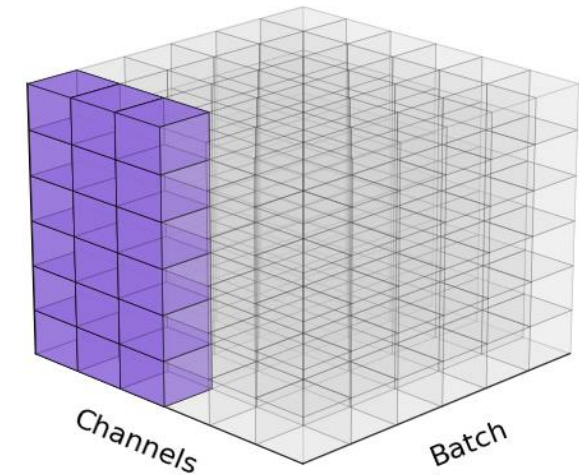


Instance Normalization

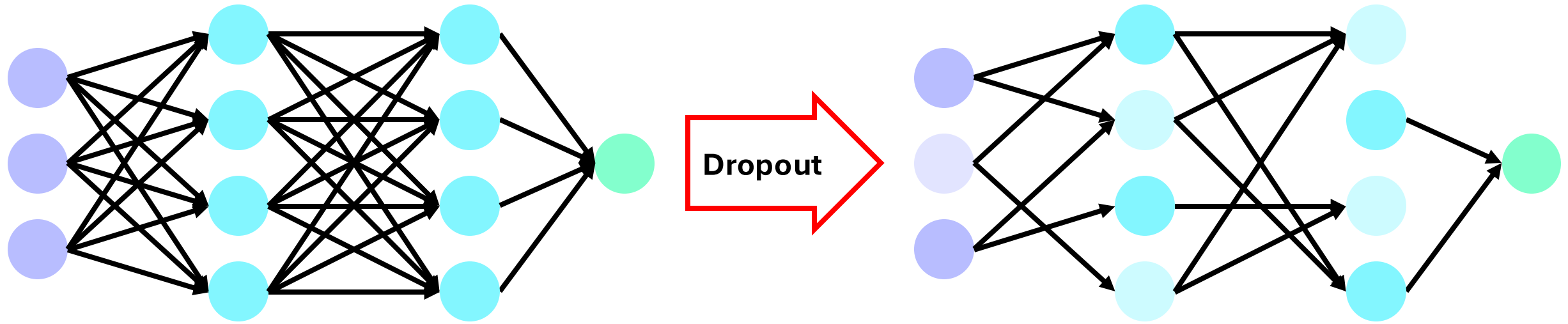


Group Normalization

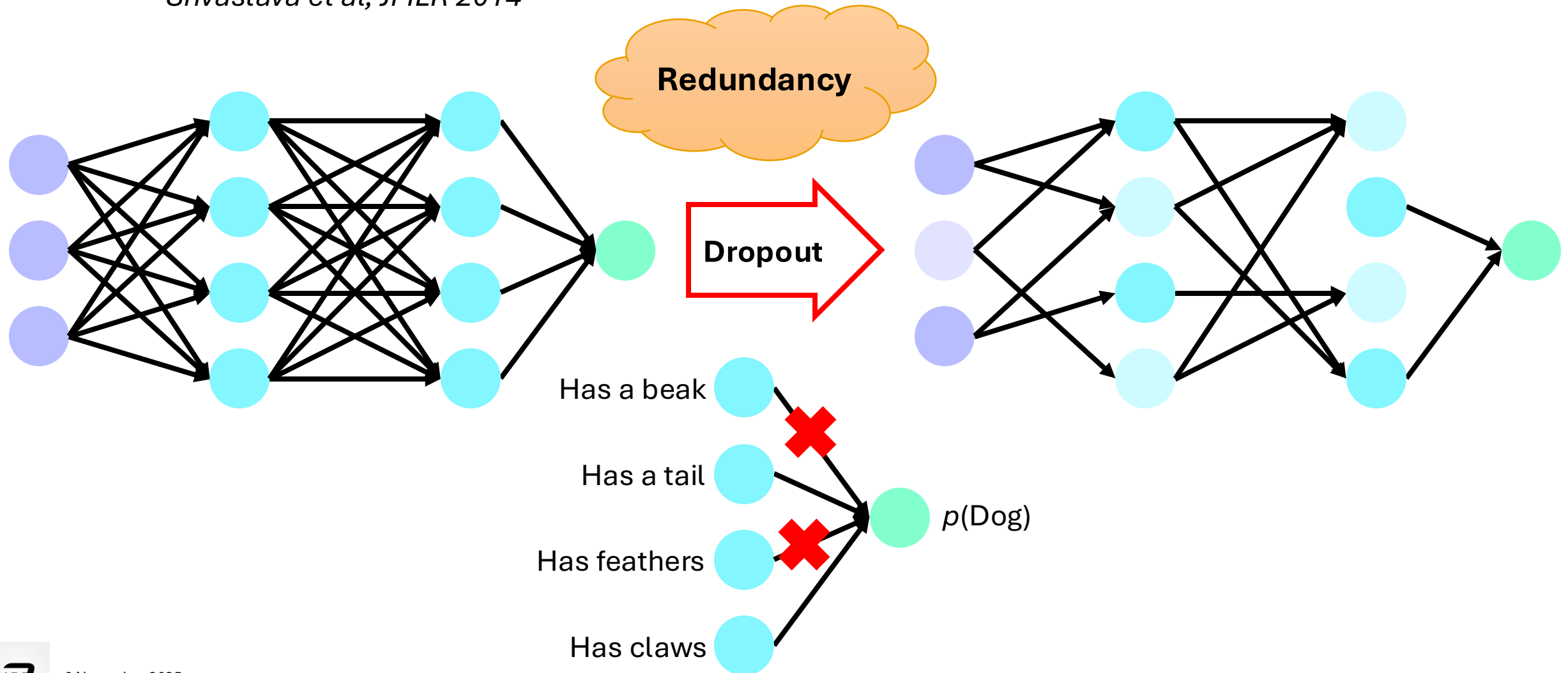
Wu & He, ECCV 2018



“Dropout: A simple way to prevent neural networks from overfitting”,
Srivastava et al, JMLR 2014



“Dropout: A simple way to prevent neural networks from overfitting”,
Srivastava et al, JMLR 2014



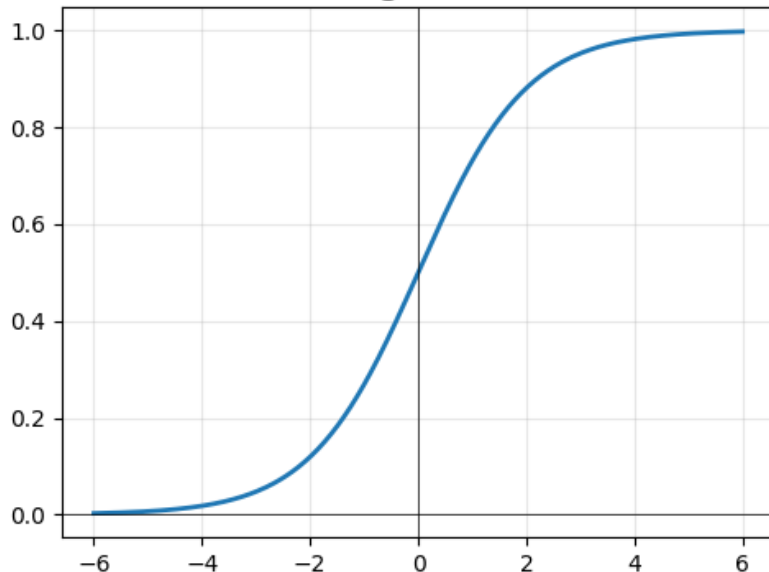
- Dropout: ensemble of several models that share parameters!
- At test (inference) all neurons and connections are active!
 - Each neuron's activation *MUST* be scaled!

```
def classify(x):  
    H1 = torch.relu(torch.matmul(W1, x) + b1) * p  
    H2 = torch.relu(torch.matmul(W2, H1) + b2) * p  
    output = torch.matmul(W3, H2) + b3  
    return output
```

- ✓ Squashes to range [0,1]
- ✓ Well-known as neuron firing rate
- x Large values **kill** the gradient
- x Too many layers of sigmoid results in **gradient-vanishing!**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid



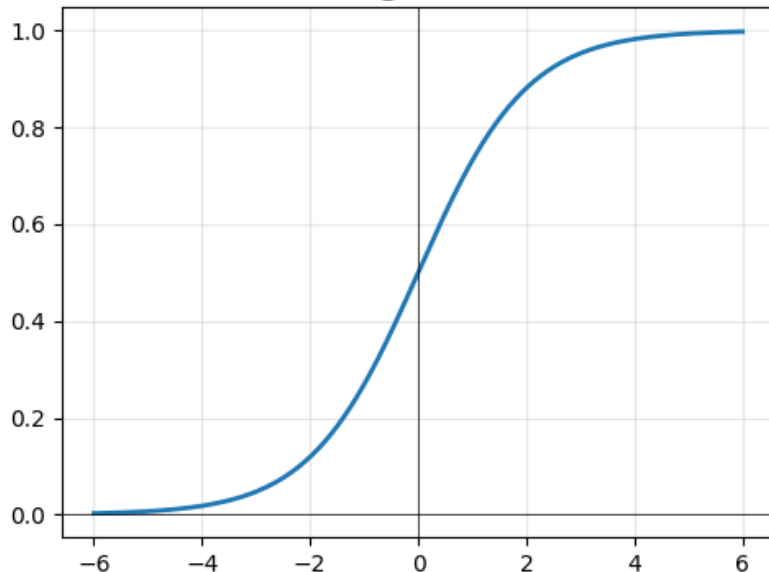
- ✓ Squashes to range $[0, 1]$
- ✓ Well-known as neuron firing rate
- x Large values **kill** the gradient
- x Too many layers of sigmoid results in **gradient-vanishing**!

- Between $[-1, 1]$ and zero-centered
- Slightly slower gradient-vanishing
- Still saturates like Sigmoid does

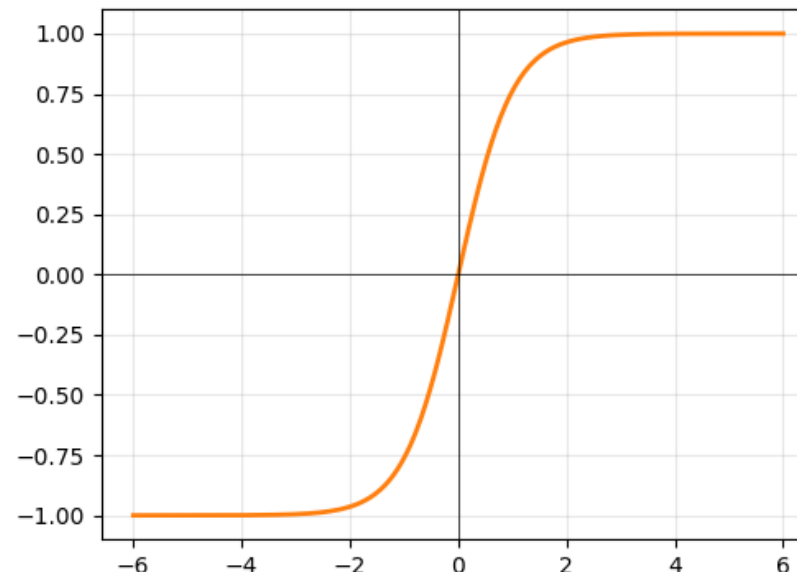
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Sigmoid

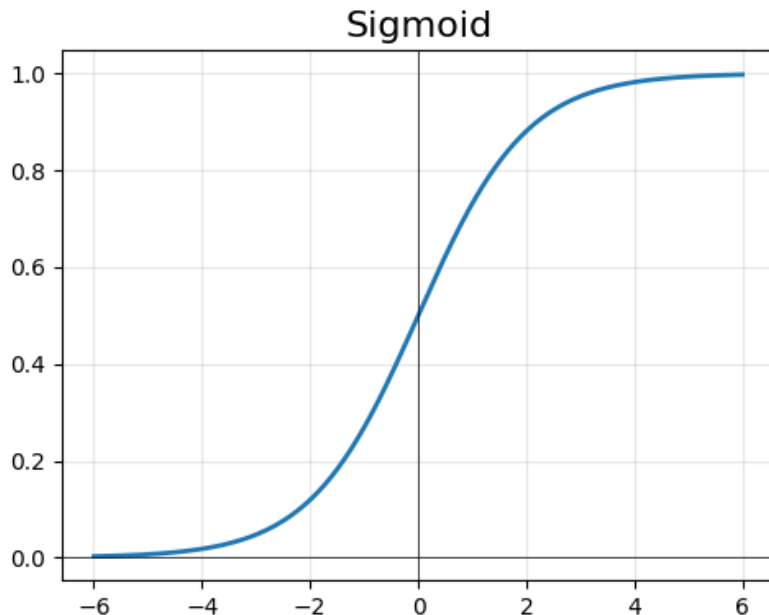


tanh



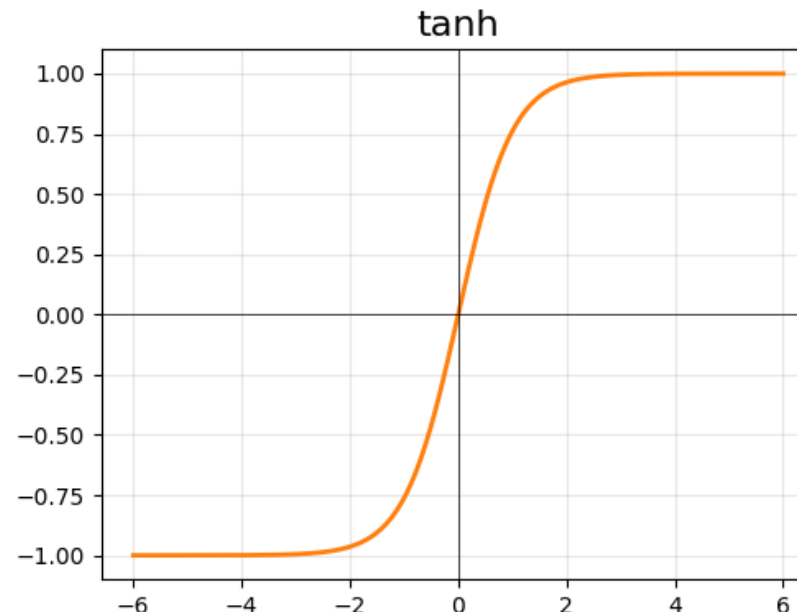
- ✓ Squashes to range $[0, 1]$
- ✓ Well-known as neuron firing rate
- x Large values **kill** the gradient
- x Too many layers of sigmoid results in **gradient-vanishing**!

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



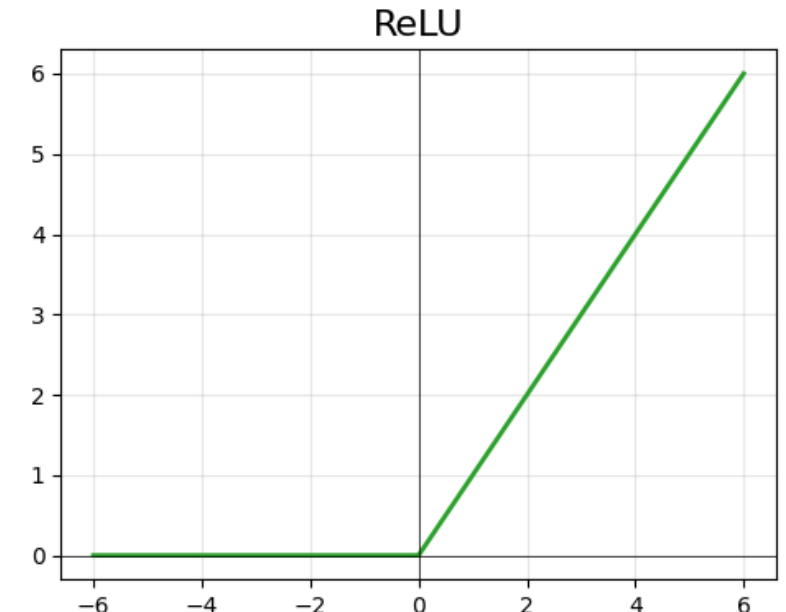
- Between $[-1, 1]$ and zero-centered
- Slightly slower gradient-vanishing
- Still saturates like Sigmoid does

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



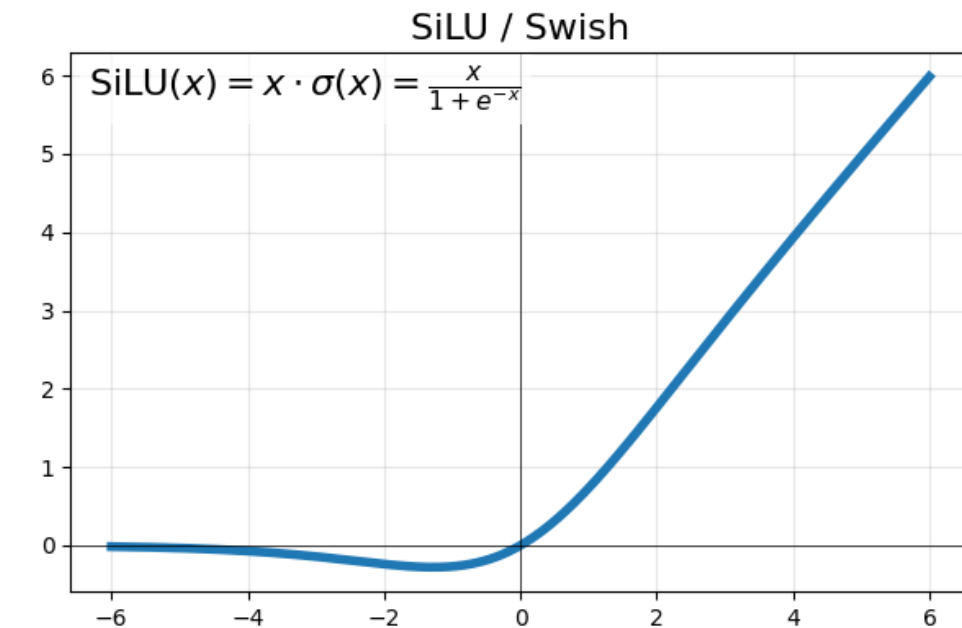
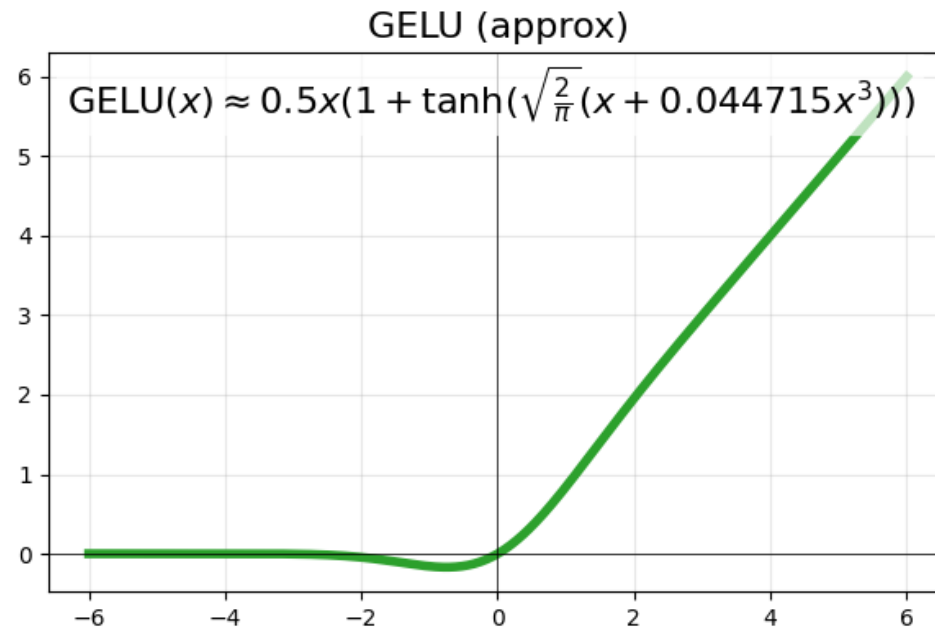
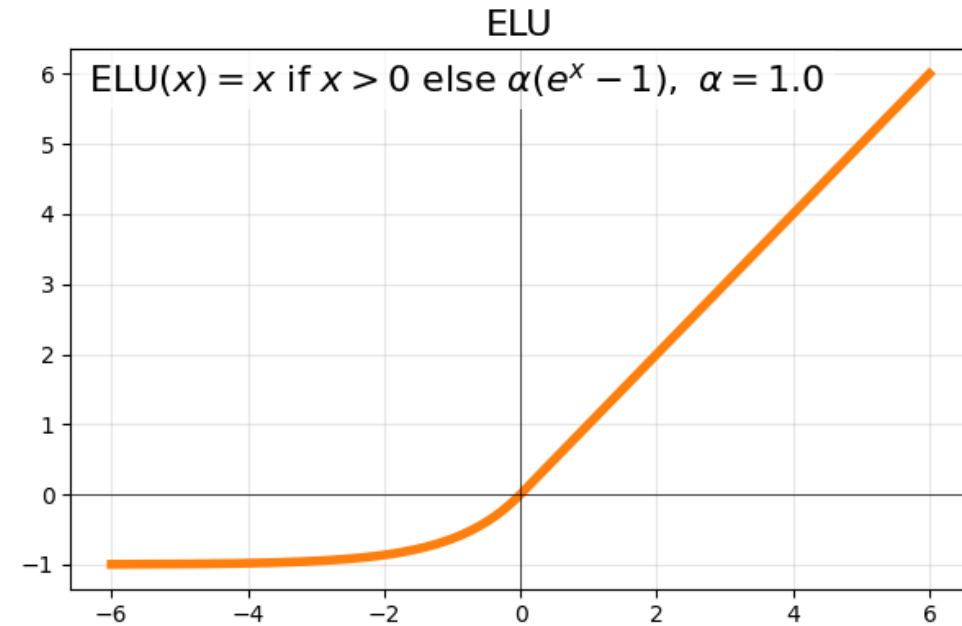
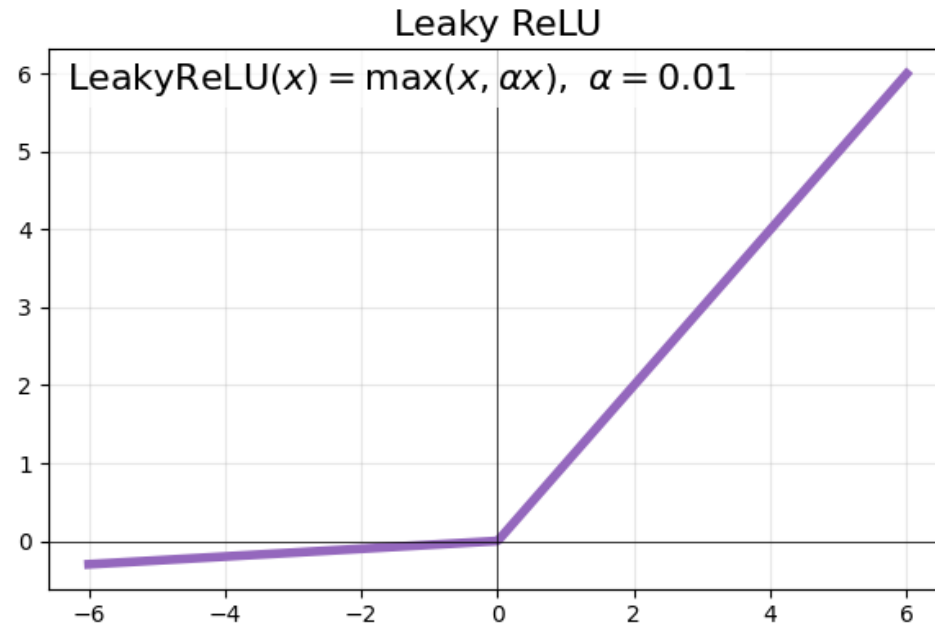
- ✓ Does not saturate
- ✓ Computationally efficient
- ✓ Converges much faster ($\sim 6x$)
- x Not zero-centered
- x Dead zone for negatives

$$\text{ReLU}(x) = \max(0, x)$$



Training Deep Neural Networks

Activation Functions (Non-linearity)



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners (2012-present):
AlexNet, VGG, GoogleNet, ResNet, DenseNet, AllConvNet, etc.

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners (2012-present):
AlexNet, VGG, GoogleNet, ResNet, DenseNet, AllConvNet, etc.

AlexNet Krizhevsky et al. 2012	VGG (11, 13, 16, 19) Simonyan & Zisserman 2014	ResNet (18, 34, 50, 101, 152) He et al. 2015

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners (2012-present):
AlexNet, VGG, GoogleNet, ResNet, DenseNet, AllConvNet, etc.

AlexNet	VGG (11, 13, 16, 19)	ResNet (18, 34, 50, 101, 152)
Krizhevsky et al. 2012	Simonyan & Zisserman 2014	He et al. 2015
227x227x3 8 Layers: 5 Conv, 3 FC 96x11x11 256x5x5 384x3x3 (2) 256x3x3 4096 4096 1000 ReLU + Softmax Dropout 0.5 ~60M trainable 2 branches, 2 GPUs		

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners (2012-present):
AlexNet, VGG, GoogleNet, ResNet, DenseNet, AllConvNet, etc.

AlexNet	VGG (11, 13, 16, 19)					ResNet (18, 34, 50, 101, 152)
Krizhevsky et al. 2012	Simonyan & Zisserman 2014					He et al. 2015
227x227x3	224x224x3	224x224x3	224x224x3	224x224x3		
8 Layers:	11 Layers:	13 Layers:	16 Layers:	19 Layers:		
5 Conv, 3 FC	8 Conv, 3 FC	10 Conv, 3 FC	13 Conv, 3 FC	16 Conv, 3 FC		
96x11x11	64x3x3	64x3x3 (2)	64x3x3 (2)	64x3x3 (2)		
256x5x5	128x3x3	128x3x3 (2)	128x3x3 (2)	128x3x3 (2)		
384x3x3 (2)	256x3x3 (2)	256x3x3 (2)	256x3x3 (3)	256x3x3 (4)		
256x3x3	512x3x3 (2)	512x3x3 (2)	512x3x3 (3)	512x3x3 (4)		
4096	512x3x3 (2)	512x3x3 (2)	512x3x3 (3)	512x3x3 (4)		
4096	4096	4096	4096	4096		
1000	4096	4096	4096	4096		
ReLU + Softmax	1000	1000	1000	1000		
Dropout 0.5	ReLU, Softmax	ReLU, Softmax	ReLU, Softmax	ReLU, Softmax		
~60M trainable	Dropout 0.5	Dropout 0.5	Dropout 0.5	Dropout 0.5		
2 branches, 2 GPUs	<133M trainable	>133M trainable	~138M trainable	~144M trainable		

Training Deep Neural Networks

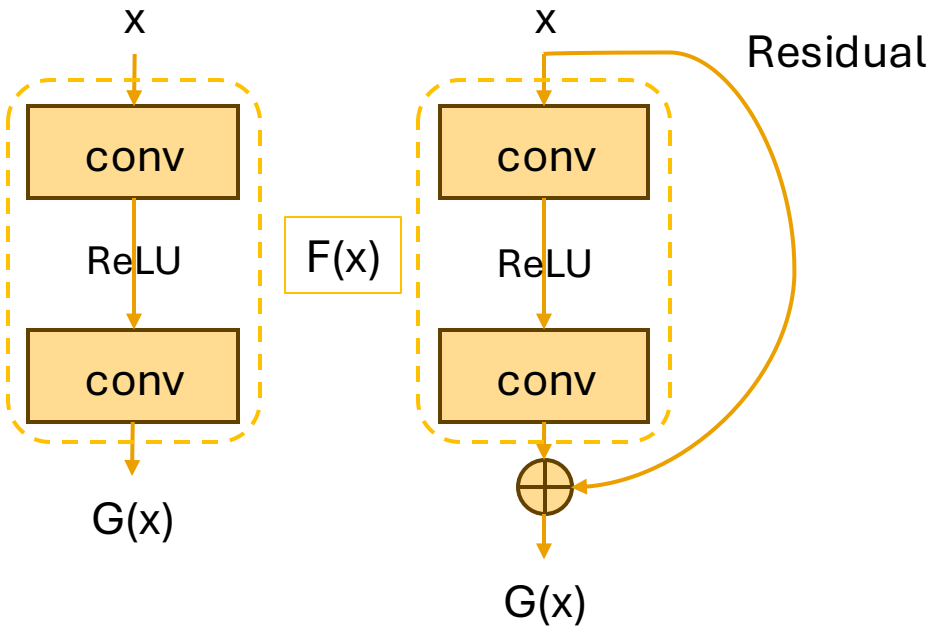
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners (2012-present):
AlexNet, VGG, GoogleNet, ResNet, DenseNet, AllConvNet, etc.

AlexNet	VGG (11, 13, 16, 19)				
Krizhevsky et al. 2012	Simonyan & Zisserman 2014				
227x227x3	224x224x3	224x224x3	224x224x3	224x224x3	
8 Layers:	11 Layers:	13 Layers:	16 Layers:	19 Layers:	
5 Conv, 3 FC	8 Conv, 3 FC	10 Conv, 3 FC	13 Conv, 3 FC	16 Conv, 3 FC	
96x11x11	64x3x3	64x3x3 (2)	64x3x3 (2)	64x3x3 (2)	
256x5x5	128x3x3	128x3x3 (2)	128x3x3 (2)	128x3x3 (2)	
384x3x3 (2)	256x3x3 (2)	256x3x3 (2)	256x3x3 (3)	256x3x3 (4)	
256x3x3	512x3x3 (2)	512x3x3 (2)	512x3x3 (3)	512x3x3 (4)	
4096	512x3x3 (2)	512x3x3 (2)	512x3x3 (3)	512x3x3 (4)	
4096	4096	4096	4096	4096	
1000	4096	4096	4096	4096	
ReLU + Softmax	1000	1000	1000	1000	
Dropout 0.5	ReLU, Softmax	ReLU, Softmax	ReLU, Softmax	ReLU, Softmax	
~60M trainable	Dropout 0.5	Dropout 0.5	Dropout 0.5	Dropout 0.5	
2 branches, 2 GPUs	<133M trainable	>133M trainable	~138M trainable	~144M trainable	

ResNet (18, 34, 50, 101, 152)

He et al. 2015

- ResNet architecture:
- Stack residual blocks
 - Every residual block has two 3x3 conv layers



Training Deep Neural Networks

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners (2012-present):
AlexNet, VGG, GoogleNet, ResNet, DenseNet, AllConvNet, etc.

AlexNet	VGG (11, 13, 16, 19)				
Krizhevsky et al. 2012	Simonyan & Zisserman 2014				
227x227x3	224x224x3	224x224x3	224x224x3	224x224x3	
8 Layers:	11 Layers:	13 Layers:	16 Layers:	19 Layers:	
5 Conv, 3 FC	8 Conv, 3 FC	10 Conv, 3 FC	13 Conv, 3 FC	16 Conv, 3 FC	
96x11x11	64x3x3	64x3x3 (2)	64x3x3 (2)	64x3x3 (2)	
256x5x5	128x3x3	128x3x3 (2)	128x3x3 (2)	128x3x3 (2)	
384x3x3 (2)	256x3x3 (2)	256x3x3 (2)	256x3x3 (3)	256x3x3 (4)	
256x3x3	512x3x3 (2)	512x3x3 (2)	512x3x3 (3)	512x3x3 (4)	
4096	512x3x3 (2)	512x3x3 (2)	512x3x3 (3)	512x3x3 (4)	
4096	4096	4096	4096	4096	
1000	4096	4096	4096	4096	
ReLU + Softmax	1000	1000	1000	1000	
Dropout 0.5	ReLU, Softmax	ReLU, Softmax	ReLU, Softmax	ReLU, Softmax	
~60M trainable	Dropout 0.5	Dropout 0.5	Dropout 0.5	Dropout 0.5	
2 branches, 2 GPUs	<133M trainable	>133M trainable	~138M trainable	~144M trainable	

ResNet (18, 34, 50, 101, 152)

He et al. 2015

- ResNet architecture:
- Stack residual blocks
 - Every residual block has two 3x3 conv layers

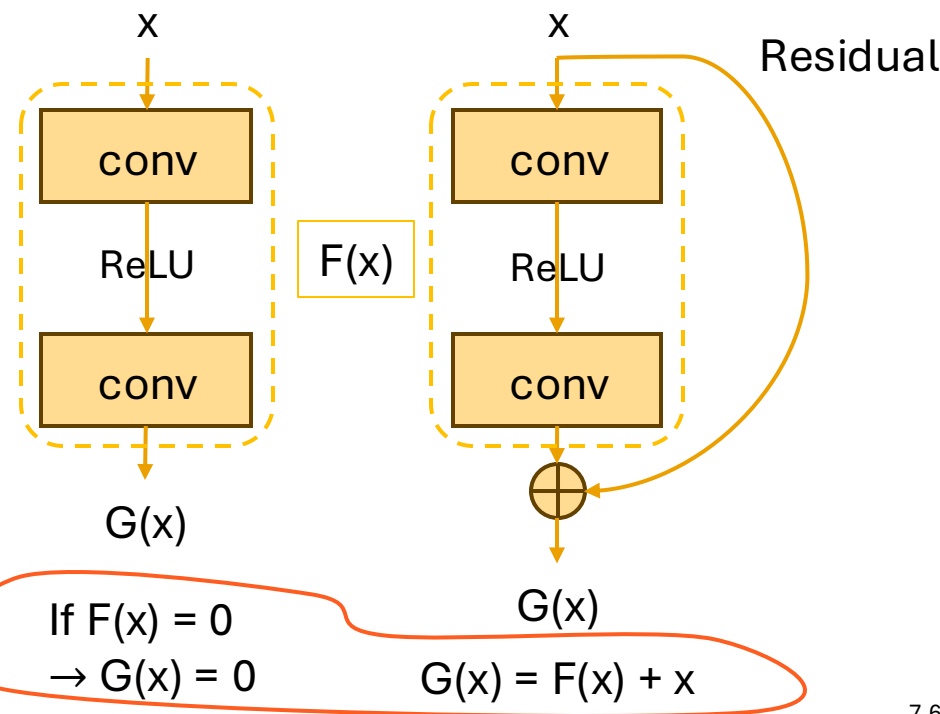


Image (input) Normalization: Center and scale for each channel

- Subtract per-channel mean and divide by per-channel std
- Requires pre-calculation of mean and std for each pixel channel

```
import numpy as np
for pixel in images: # dataset
    for i in range(pixel.shape[0]):
        for j in range(pixel.shape[1]):
            for c in range(pixel.shape[2]):
                norm_pixel[i,j,c] = (pixel[i,j,c] - np.mean(pixel[:, :, c])) / np.std(pixel[:, :, c])
```

Image (input) Normalization: Center and scale for each channel

- Subtract per-channel mean and divide by per-channel std
- Requires pre-calculation of mean and std for each pixel channel

```
import numpy as np
for pixel in images: # dataset
    for i in range(pixel.shape[0]):
        for j in range(pixel.shape[1]):
            for c in range(pixel.shape[2]):
                norm_pixel[i,j,c] = (pixel[i,j,c] - np.mean(pixel[:, :, c])) / np.std(pixel[:, :, c])
```

Image (input) Resizing/Cropping:

- Deep learning models expect fixed input sizes: e.g. 224x224

```
import torch
from torchvision import transforms
transforms.Resize((224, 224))
```

Regularization: A common pattern

- While training: add some randomness $y = fw(x, z)$
- At test: average out randomness $y = f(x) = E_z[f(x, z)] = \int p(z) f(x, z) dz$

Training Deep Neural Networks

Data Augmentation

Regularization: A common pattern

- While training: add some randomness $y = fw(x, z)$
- At test: average out randomness $y = f(x) = E_z[f(x, z)] = \int p(x)f(x, z)dz$



Dropout: randomly drop activations

Training Deep Neural Networks

Data Augmentation

Regularization: A common pattern

- While training: add some randomness
- At test: average out randomness

$$y = fw(x, z)$$

$$y = f(x) = E_z[f(x, z)] = \int p(z) f(x, z) dz$$

Dropout: randomly
drop activations

Use all activations
and average with p

Training Deep Neural Networks

Data Augmentation

Regularization: A common pattern

- While training: add some randomness
- At test: average out randomness

$$y = fw(x, z)$$

$$y = f(x) = E_z[f(x, z)] = \int p(x)f(x, z)dz$$

Dropout: randomly
drop activations

Augmentation:

Use all activations
and average with p



Training Deep Neural Networks

Data Augmentation

Regularization: A common pattern

- While training: add some randomness
- At test: average out randomness

$$y = fw(x, z)$$

$$y = f(x) = E_z[f(x, z)] = \int p(z) f(x, z) dz$$

Dropout: randomly drop activations

Augmentation:

- Random contrast and brightness

Use all activations and average with p



Training Deep Neural Networks

Data Augmentation

Regularization: A common pattern

- While training: add some randomness
- At test: average out randomness

$$y = fw(x, z)$$

$$y = f(x) = E_z[f(x, z)] = \int p(x)f(x, z)dz$$

Dropout: randomly drop activations

Augmentation:

- Random contract and brightness
- Random cropping and scaling

Use all activations and average with p



Training Deep Neural Networks

Data Augmentation

Regularization: A common pattern

- While training: add some randomness
- At test: average out randomness

$$y = fw(x, z)$$

$$y = f(x) = E_z[f(x, z)] = \int p(x)f(x, z)dz$$

Dropout: randomly
drop activations

Augmentation:

- Random contract and brightness
- Random cropping and scaling
- Random flipping

Use all activations
and average with p



Training Deep Neural Networks

Data Augmentation

Regularization: A common pattern

- While training: add some randomness
- At test: average out randomness

$$y = fw(x, z)$$

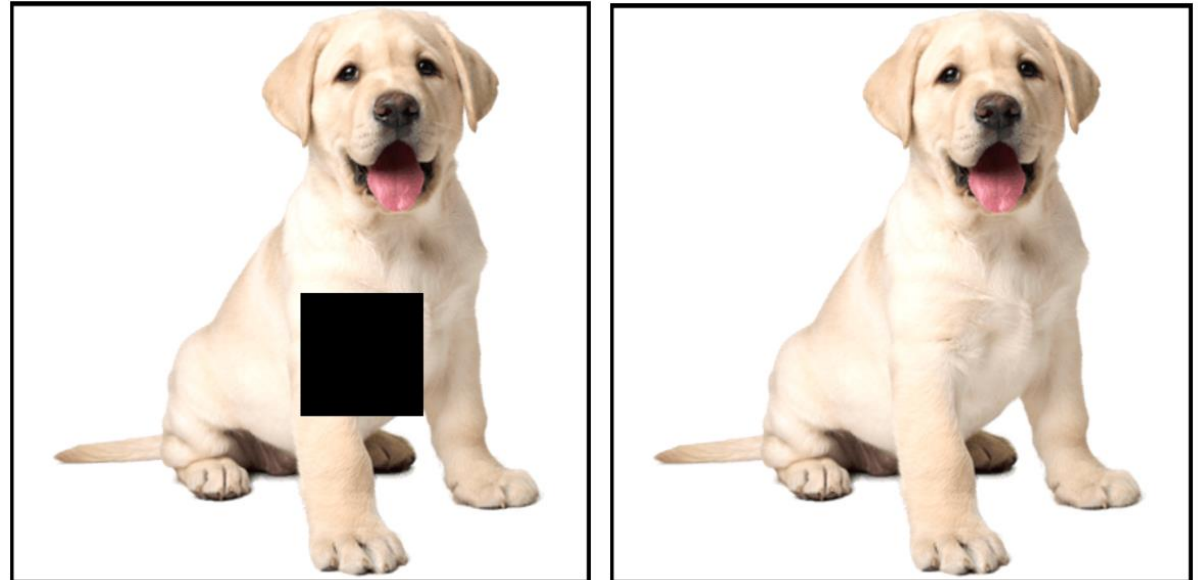
$$y = f(x) = E_z[f(x, z)] = \int p(z) f(x, z) dz$$

Dropout: randomly drop activations

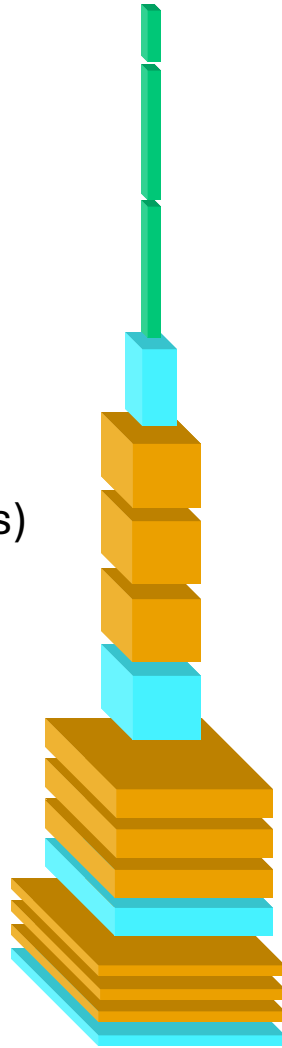
Augmentation:

- Random contrast and brightness
- Random cropping and scaling
- Random flipping
- Random cutouts (small datasets)

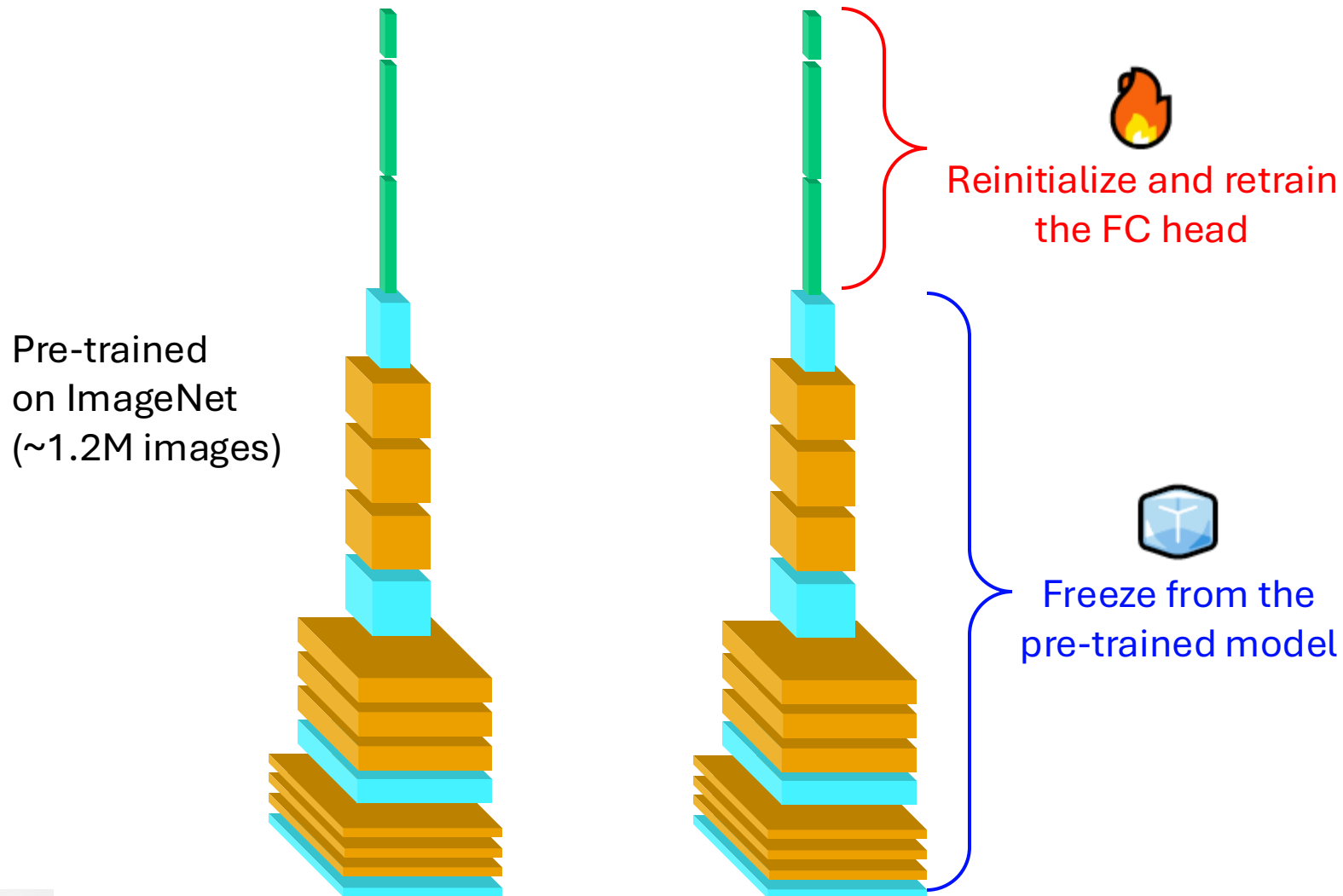
Use all activations and average with p



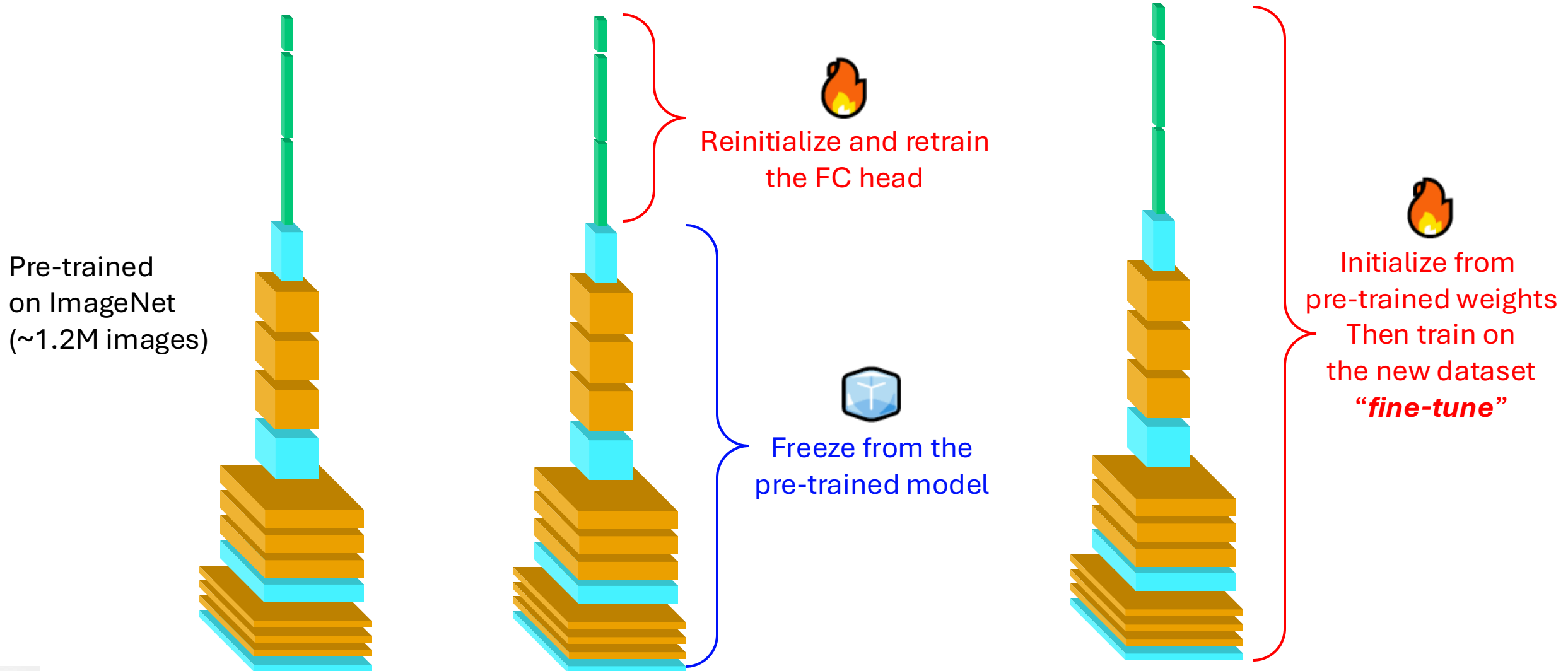
Training a deep network on a smaller dataset!



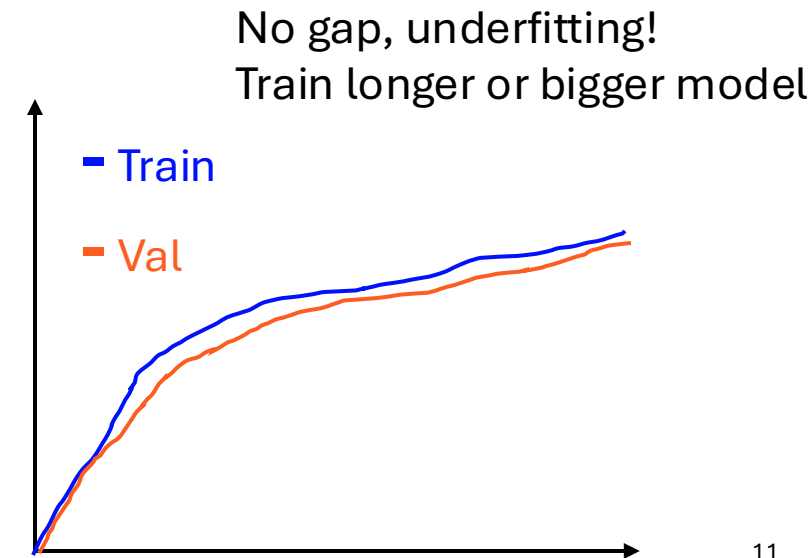
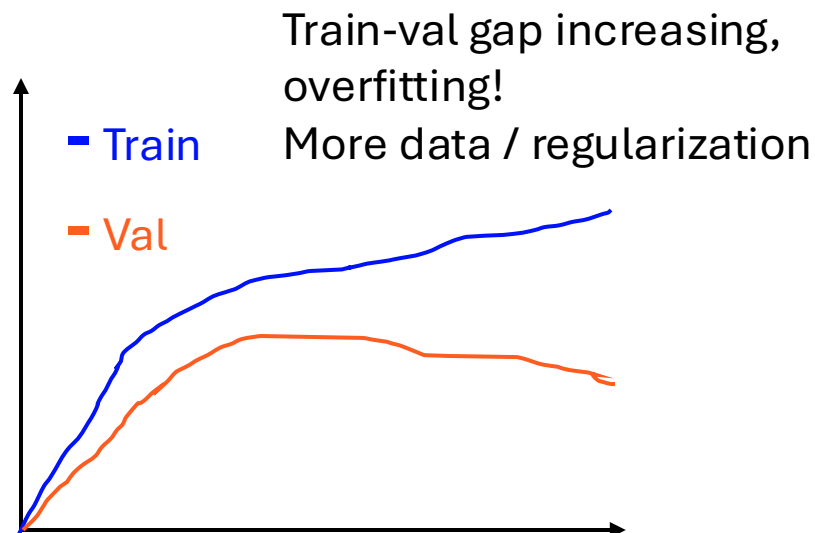
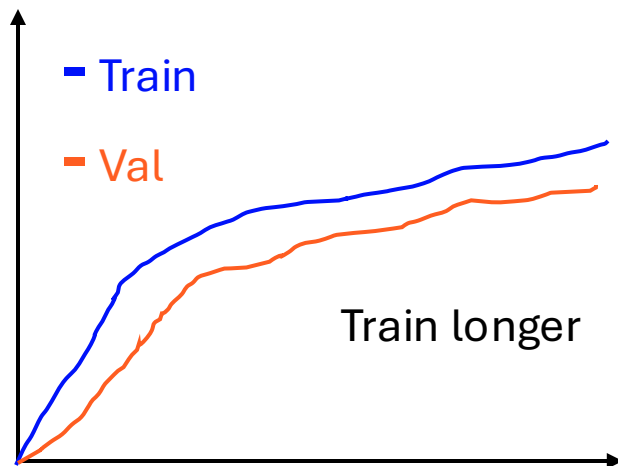
Training a deep network on a smaller dataset!



Training a *deep* network on a *smaller* dataset! But if a *large* dataset ...



1. Check initial loss
2. Overfit a small sample
3. Find a *learning-rate* that makes loss decrease significantly
 1. Common rates: $1e-1$, ... , $1e-5$
4. Try a broad set of hyperparameters, and train for 1~5 epochs
5. Refine your selection set and train longer
6. Always keep an eye on loss and accuracy curves → Step 5.







(Q&A)